GaussianCity: Generative Gaussian Splatting for Unbounded 3D City Generation

Taro Yamada
Anonymous Institution

Abstract

Generating photorealistic 3D cities with unbounded spatial extent is important for simulation, virtual production, and immersive navigation, but existing urban scene generators are dominated by neural radiance fields whose rendering cost grows unfavorably with scene size. We present GaussianCity, a generative city model that brings feed-forward 3D Gaussian splatting to unbounded urban generation through a compact, visibility-aware BEV-Point representation and a spatially aware point decoder. The key idea is to keep geometry explicit and sparse by extruding BEV height and semantic maps into points, pruning them using a density map and camera-aware ray intersection, and then recovering appearance with a decoder that combines BEV conditioning, point serialization, Point Transformer blocks, and per-instance style modulation. The resulting model preserves the efficiency advantages of Gaussian rasterization while avoiding the prohibitive point counts of naive city-scale 3D-GS. On GoogleEarth, GaussianCity achieves an FID of 86.94, KID of 0.090, DE of 0.136, and CE of 0.057 while rendering in 93 ms per view, about 60 times faster than CityDreamer. On KITTI-360, it improves over UrbanGIRAFFE from 39.6 to 29.5 FID and from 0.036 to 0.017 KID. Ablations further show that visibility-aware BEV-Point initialization reduces the active point set from tens of millions to 31.7K points, and that the proposed decoder is essential for recovering image quality from this compact representation.

1. Introduction

Unbounded 3D city generation is a challenging setting for controllable scene synthesis. The model must preserve large-scale layout, render coherent views under substantial camera motion, and recover the high-frequency appearance of roads, roofs, facades, vegetation, and distant structures. Existing urban generators based on neural radiance fields or compositional neural feature fields have shown promising realism, but they remain expensive at inference time and are difficult to scale to city-sized scenes without substantial rendering overhead [5, 14, 15, 19, 21]. This bottleneck is especially visible in interactive applications, where rendering speed matters as much as perceptual fidelity.

3D Gaussian Splatting (3D-GS) offers an attractive alternative because it replaces volumetric ray marching with fast rasterization over explicit Gaussians [10]. However, directly porting 3D-GS to unbounded city generation is not straightforward. If one instantiates dense Gaussians throughout a large urban volume, the representation quickly becomes impractical: memory usage grows with scene area, most primitives are not visible from a given camera, and unordered point sets make appearance decoding difficult. In other words, Gaussian rendering is fast, but naive Gaussian parameterization is not scalable.

GaussianCity addresses this mismatch with two design choices. First, we introduce *BEV-Point*, a compact point representation derived from bird's-eye-view (BEV) projections. BEV-Point decouples large-scale geometry from view-dependent rendering: points are initialized by extruding BEV height and semantic maps, thinned by a density map, and then aggressively pruned by ray-based visibility so that only view-relevant points remain. This keeps memory nearly constant with respect to the latent city extent visible in the rasterizer. Second, because aggressive pruning destroys the regular structure of the BEV grid, we introduce a BEV-Point decoder that re-injects spatial order into the unordered points using serialization and point attention. The decoder operates on explicit point geometry, BEV scene features, relative coordinates, and per-instance style codes, and predicts Gaussian appearance for rasterization.

The overall framework is shown in Figure 1. GaussianCity routes points to category-specific generators for background regions, buildings, and, when present, vehicles, and renders the resulting Gaussians with a differentiable rasterizer. Across both drone-view and street-view benchmarks, the method consistently improves quality or efficiency over prior urban generators. The compact representation is not only a systems optimization; it is a modeling choice that makes feed-forward Gaussian city generation feasible at all.

Our contributions are threefold:

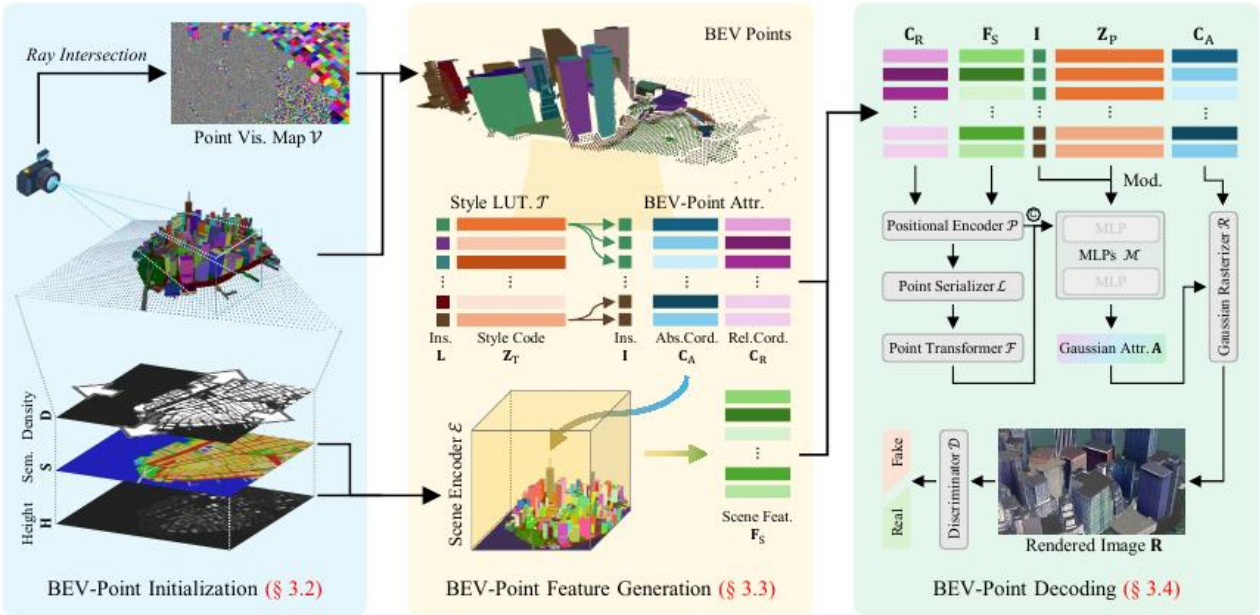

Figure 1. **Framework of GaussianCity.** BEV semantic, height, and density projections are extruded into 3D points, pruned by ray-based visibility, enriched with scene and instance features, and decoded into Gaussian appearance for differentiable rasterization.

- We present, to our knowledge, the first generative framework that uses 3D Gaussian splatting for unbounded city-scale scene generation rather than reconstruction.
- We introduce BEV-Point, a visibility-aware city representation that drastically reduces active point count and keeps the rendering footprint practical for large scenes.
- We propose a spatially aware BEV-Point decoder with point serialization, Point Transformer blocks, and instance-wise style modulation, achieving state-of-the-art fidelity with much faster inference than NeRF-based urban baselines.

2. Related Work

Generative urban scene modeling. 3D-aware scene generators such as GIRAFFE [14] and SGAM [15] established the value of compositional scene representations for controllable generation, but they target either generic scenes or virtual-world construction rather than city-scale feed-forward rendering. UrbanGIRAFFE [21] brought compositional neural feature fields to driving scenes, while SceneDreamer [5], InfiniCity [12], and CityDreamer [19] specifically studied unbounded city generation from BEV-like priors. These methods demonstrate that urban generation benefits from structured layout inputs, but their rendering backbones are still built around neural fields, which limits interactive inference speed. GaussianCity keeps the BEV conditioning paradigm of prior urban work but replaces neural field rendering with explicit Gaussians.

Gaussian representations for efficient 3D generation. 3D-GS [10] and follow-up work such as Scaffold-GS [13] show that explicit Gaussian primitives can support real-time rendering with strong fidelity. Feed-forward Gaussian models including Splatter Image [16], LGM [17], and GRM [20] further demonstrate that Gaussian attributes can be predicted directly from images. However, these methods focus on object-centric or bounded-scene reconstruction and do not address the representation problem that arises when the target scene is an unbounded city. Our method borrows the efficiency of Gaussian rasterization but introduces a city-specific point parameterization that avoids allocating Gaussians densely over massive empty or occluded space.

Point ordering and structure recovery. Once BEV points are thinned by visibility, they no longer lie on a regular grid. This makes local context modeling harder than in conventional image or voxel decoders. Point Transformer V3 [18] shows that serialized point organization is an efficient way to scale point processing, while Hilbert-curve ordering is a classical tool for locality-preserving serialization [4, 8]. GaussianCity adapts this line of work to generative city rendering: serialization is used not for downstream point recognition, but for restoring spatial coherence before Gaussian attribute prediction.

3. Method

3.1. Overview

GaussianCity takes BEV projections of an urban region and a target camera pose as input, and outputs a rendered image by predicting Gaussian appearance for a compact set of visible 3D points. The pipeline has three stages: (1) BEV-Point initialization from BEV semantic and height projections; (2) BEV-Point feature generation, including scene features, instance attributes, and per-instance style codes; and (3) BEV-Point decoding into Gaussian attributes followed by rasterization. Figure 1 summarizes the full pipeline.

3.2. Visibility-Aware BEV-Point Initialization

The representation starts from BEV projections containing semantic labels, top-down height fields, bottom-up height fields, and a binary point-validity mask. We extrude these maps into 3D points with integer-valued position, an inherited geometric scale, and an instance identifier. Intuitively, this converts the city from a dense volume into a set of semantically grounded surface samples. Buildings, roads, water, sky, and other categories are handled through category-specific scale rules, while building roofs and facades retain separate instance-aware identities.

Direct extrusion still yields too many points for city-scale rendering. GaussianCity therefore applies two pruning steps. First, a binary density map suppresses locations that do not need point support, reducing redundancy before any neural processing. Second, given the target camera, we compute a visibility map by ray-voxel intersection [1] and retain only points that are intersected by at least one ray. This is crucial: the representation should scale with what must be rendered, not with the full latent volume of the city. As shown later in Table 2, these two steps reduce the active set from tens of millions of points to 31.7K without sacrificing image quality.

Each surviving point stores absolute coordinates and a normalized coordinate within its associated instance bounding volume. For an instance with center (c_x, c_y) and size (w, h, d), the relative coordinate simply rescales the point to a canonical range within that instance. This factorization lets the decoder separate global layout from local shape and appearance.

3.3. BEV-Point Feature Generation

For every visible point, the decoder uses four sources of information.

Scene features. We encode the cropped BEV height and semantic maps with a scene encoder. Depending on the generator branch, the implementation uses either a BEV feature extractor sampled at point locations or a global BEV encoder that broadcasts scene context to all points. This gives each point access to large-scale layout and neighborhood semantics without reverting to a dense 3D representation.

Geometric features. We concatenate the scene feature with the point's relative coordinate and then apply a positional encoder. In practice, the implementation supports either hash-grid encoding or sinusoidal encoding, enabling a compact but expressive representation of location.

Instance and class information. Each point receives a one-hot semantic label derived from its instance ID. This is important in urban scenes because the same geometric position can imply different appearance statistics depending on whether the point belongs to road, facade, roof, water, or vehicle regions.

Style codes. To model appearance diversity independently of geometry, GaussianCity associates each instance with a latent code from a style lookup table. Background regions can operate without style modulation, while instance-centric branches such as buildings use learned per-instance latent vectors. The decoder then conditions its modulated MLP layers on these codes, following the spirit of style modulation in generative image models [9].

3.4. Spatially Aware BEV-Point Decoder

The central challenge is that visible points form an unordered sparse set. A plain MLP on point-wise features ignores locality and leads to inconsistent facades, roof boundaries, and object-level appearance. GaussianCity addresses this with a BEV-Point decoder built from serialization, point attention, and modulated attribute prediction.

Point serialization. We first quantize the point coordinates onto a grid and serialize them so that nearby points tend to appear close in a 1D order. Our default raster-scan serialization is

$$s(\mathbf{p}) = q_x M^2 + q_y M + q_z, \tag{1}$$

where (q_x, q_y, q_z) are quantized point coordinates and M is the quantization range per axis. We also study Hilbert-curve ordering [4, 8] and their combination in Table 4. Serialization supplies the decoder with a stable spatial order without returning to a dense voxel tensor.

Point context modeling. Serialized point features are processed by Point Transformer V3 blocks [18], which aggregate neighborhood context efficiently on the sparse visible set. This step is responsible for restoring structural coherence across points that originally came from a regular BEV representation but were broken apart by visibility filtering.

Gaussian attribute prediction. The final prediction head is a modulated MLP. In the released implementation, the model predicts Gaussian RGB values while geometry is inherited from the BEV-Point initialization: absolute point coordinates define the Gaussian centers, category-aware point scales define the Gaussian size, rotations are fixed, and opacity is set to one. This design is deliberate. Because the scene geometry already comes from the structured BEV projections, learning only the appearance is sufficient to obtain high-quality rendering while keeping the parameterization stable and memory-efficient.

Points are routed to category-specific generators. On GoogleEarth, GaussianCity uses separate generators for background regions and buildings. On KITTI-360, an additional vehicle branch can be enabled. After per-branch decoding, all Gaussians are merged and rendered with a differentiable Gaussian rasterizer [10].

3.5. Training Objective

Training is fully image-supervised. Given a target view, we render the predicted Gaussian set and optimize a weighted sum of masked pixel-wise ℓ_1 loss, perceptual loss, and adversarial loss:

$$\mathcal{L} = \lambda_1 \mathcal{L}_1 + \lambda_p \mathcal{L}_{\text{perc}} + \lambda_g \mathcal{L}_{\text{GAN}}. \tag{2}$$

The implementation uses a VGG-based perceptual loss and a semantic-aware discriminator trained on the rendered RGB image together with the semantic mask. This objective encourages both local fidelity and globally realistic texture statistics while preserving multi-view consistency through the shared 3D point representation.

4. Experiments

4.1. Experimental Setup

Datasets. We evaluate on two urban benchmarks. GoogleEarth, introduced in prior unbounded city generation work [19], provides large-scale drone-view imagery with BEV annotations and diverse city layouts. KITTI-360 [11] provides real-world street-view urban scenes with semantic and instance annotations. These two datasets stress complementary challenges: GoogleEarth emphasizes unbounded aerial generation with large visible extents, while KITTI-360 emphasizes street-level realism and perspective variation.

Implementation protocol. The implementation trains with batch size 1 for 500 epochs using image reconstruction, perceptual, and adversarial supervision. For GoogleEarth, the data loader covers 400 cities with 60 views per city in the released configuration, and the model is trained with separate background and building generators. KITTI-360 follows the same rendering pipeline but uses the street-view camera model and can additionally include a vehicle branch. Runtime in Table 1 is reported in milliseconds on an NVIDIA Tesla A100, following the provided evaluation table.

Metrics. We report Fréchet Inception Distance (FID) [7] and Kernel Inception Distance (KID) [2]. For GoogleEarth, we also report DE and CE following the evaluation protocol of CityDreamer [19]. Lower is better for all metrics.

4.2. Comparison with Prior Work

Table 1 compares GaussianCity with prior urban generation methods. On GoogleEarth, GaussianCity achieves the best score on every reported metric, improving over the strongest baseline CityDreamer from 97.38 to 86.94 FID and from 0.096 to 0.090 KID, while also reducing DE from 0.147 to 0.136 and CE from 0.060 to 0.057. The efficiency gain is even more pronounced: the runtime drops from 5580 ms to 93 ms, corresponding to roughly 60× faster inference. This gap is exactly the regime where replacing neural field rendering with Gaussian rasterization matters in practice.

On KITTI-360, GaussianCity also performs best, lowering FID from 39.6 to 29.5 and KID from 0.036 to 0.017 relative to UrbanGIRAFFE. Notably, the improvements are obtained despite the street-view setting being visually more constrained and geometrically less forgiving than aerial generation. This suggests that the BEV-Point representation is not tied to a single camera regime.

The qualitative comparisons in Figures 3 and 4 are consistent with the numerical results. On GoogleEarth, GaussianCity renders more regular building geometry and cleaner textures than Pers.Nature, SceneDreamer, InfiniCity, and CityDreamer. On KITTI-360, it produces more realistic urban appearance than GSN and UrbanGIRAFFE, especially on buildings and road-adjacent structures. The scalability trend in Figure 2 further clarifies why: once the representation is built around a compact visible point set, Gaussian rasterization can remain efficient even as the underlying city extent grows.

4.3. Ablation on BEV-Point Representation

Table 2 isolates the effect of representation design. Without visibility pruning or density thinning, the point cloud contains 20–31 million points and the system runs out of memory. Even with density filtering, region-level visibility still leaves 1.4M points, which remains impractical. Instance-level visibility lowers the active set to 680K points and makes rendering feasible, but the ray-intersection formulation is substantially better: it reduces the point count to 38.0K without density filtering and to 31.7K with density filtering, yielding just 1.39 GB VRAM usage and 0.09 s inference time.

Table 1. **Quantitative comparison on GoogleEarth and KITTI-360.** The best results are highlighted in bold. Note that "RT." denotes "Runtime", measured in milliseconds on an NVIDIA Tesla A100.

GoogleEarth						KITTI-360		
Method	FID ↓	KID ↓	DE ↓	CE ↓	RT. ↓	Method	FID ↓	KID ↓
SGAM [15]	277.64	0.358	0.575	239.291	193	StyleGAN [9]	31.9	0.021
Pers.Nature [3]	123.83	0.109	0.326	86.371	167	GSN [6]	160.0	0.114
SceneDreamer [5]	213.56	0.216	0.152	0.186	620	GIRAFFE [14]	112.1	0.117
CityDreamer [19]	97.38	0.096	0.147	0.060	5580	UrbanGIR. [21]	39.6	0.036
GaussianCity	**86.94**	**0.090**	**0.136**	**0.057**	**93**	GaussianCity	**29.5**	**0.017**

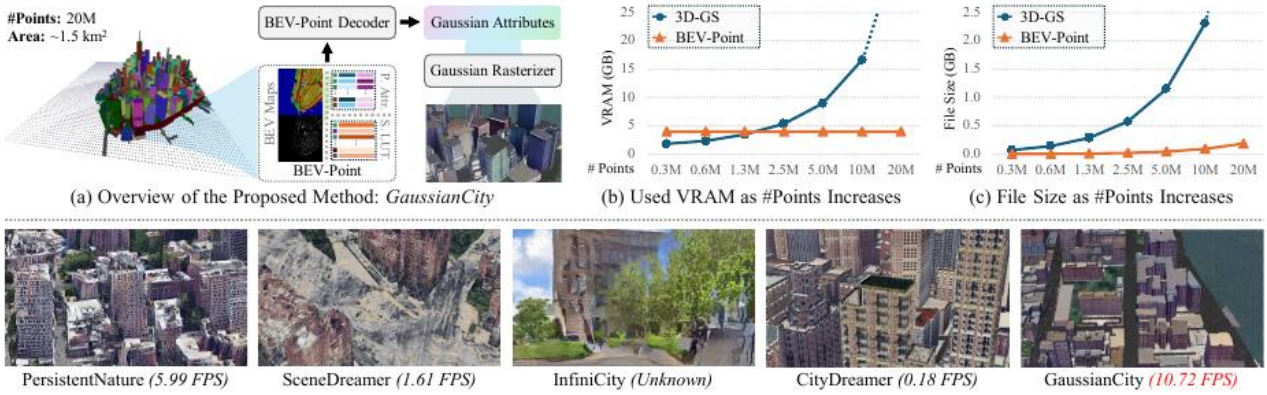

Figure 2. **Teaser and scalability analysis.** Beyond qualitative comparisons on GoogleEarth, the figure shows that naive 3D-GS memory and file size grow rapidly with point count, whereas the proposed BEV-Point representation remains compact enough for unbounded city rendering.

Table 2. **Effectiveness of BEV-Point Representation.** Note that "RayInt." denotes "Ray Intersection", respectively. The VRAM usage and inference time are measured on an NVIDIA Tesla A100.

Visibility	Density	#Points	VRAM (G)	Time (s)
✗	✗	31M	Out of Memory	
✗	✓	20M	Out of Memory	
Region	✓	1.4M	Out of Memory	
Instance	✓	680K	10.34	0.16
RayInt.	✗	38.0K	1.41	0.10
RayInt.	✓	**31.7K**	**1.39**	**0.09**

These numbers support the main systems claim of the paper. The challenge in city-scale Gaussian generation is not only how to decode appearance, but how to decide which Gaussians should exist at all. Visibility-aware BEV-Point initialization provides that answer and is the main reason the method remains scalable.

Table 3. **Effectiveness of BEV-Point Decoder.** Note that "Ser." and "Pt.Trans." denote "Point Serializer" and "Point Transformer", respectively.

Ser.	Pt.Trans.	FID ↓	KID ↓	DE ↓	CE ↓
✗	✗	151.27	0.179	0.185	0.135
✗	✓	119.40	0.138	0.159	0.118
✓	✓	**86.94**	**0.090**	**0.136**	**0.057**

Table 4. **Comparison of different serialization methods.** Note that "Eq. 1" denote the serialization used in Point Serializer.

Eq. 1	Hilbert	FID ↓	KID ↓	DE ↓	CE ↓
✓	✗	86.94	0.090	0.136	0.057
✗	✓	86.72	0.088	0.136	0.056
✓	✓	**86.28**	**0.083**	**0.135**	**0.055**

4.4. Ablation on BEV-Point Decoder

Table 3 shows that point-wise decoding alone is insufficient. Removing both serialization and point attention yields a

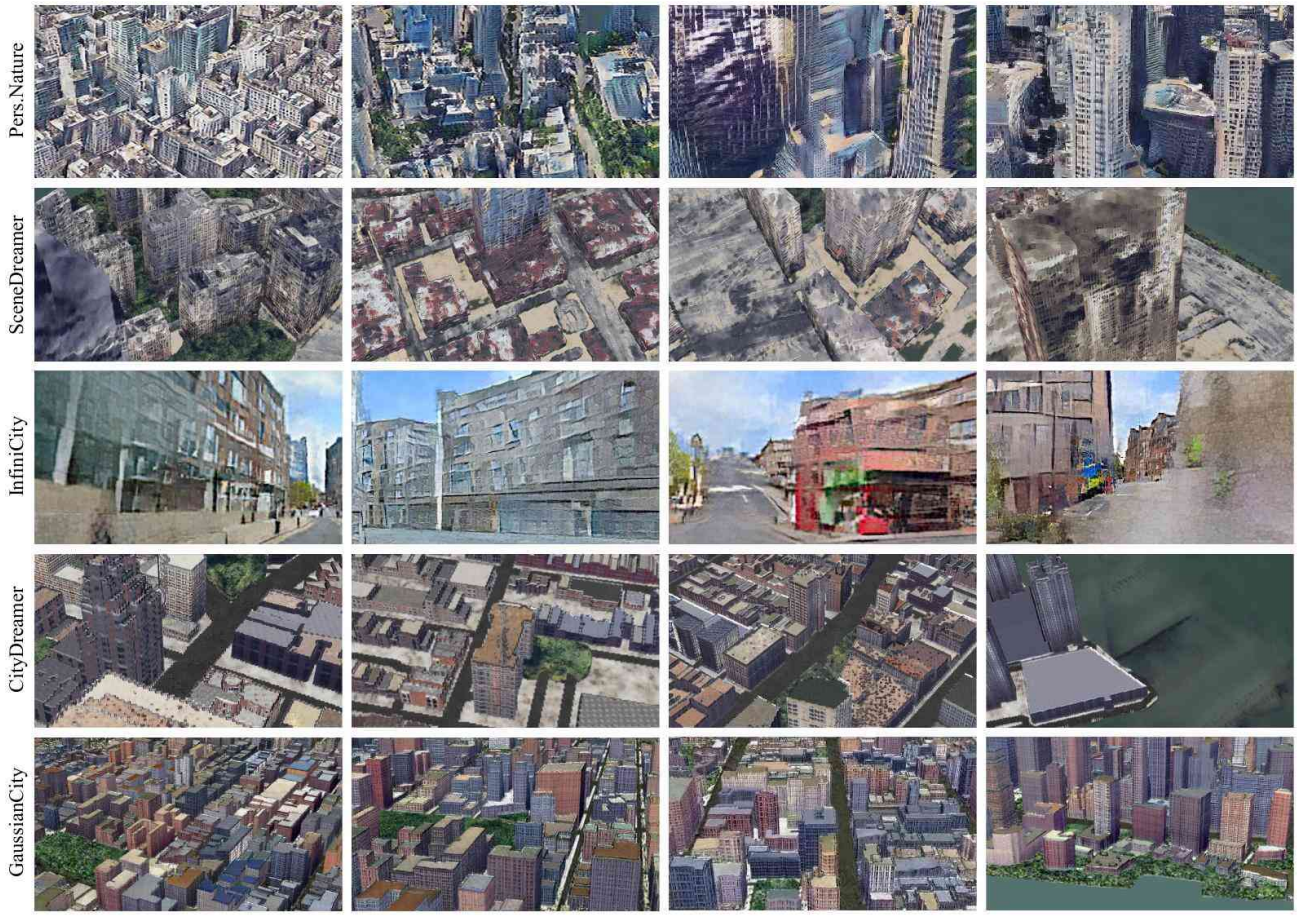

Figure 3. **Qualitative comparison on GoogleEarth.** GaussianCity produces sharper roofs, cleaner road structures, and more coherent large-scale appearance than prior urban generators.

poor FID of 151.27 and CE of 0.135, indicating that the compact representation by itself does not guarantee realistic rendering. Adding Point Transformer blocks improves FID to 119.40, confirming that contextual aggregation matters. The full decoder, which combines serialization and Point Transformer processing, reaches 86.94 FID and 0.057 CE. This is a large improvement, especially because the Gaussian geometry is fixed and only appearance is learned.

Table 4 compares different serialization strategies. The default serialization in Equation (1) and Hilbert ordering perform similarly on their own, while combining both gives the best overall result, improving FID from 86.94 to 86.28 and KID from 0.090 to 0.083. The gains are modest compared with the larger jump from enabling serialization at all, which suggests that the critical factor is restoring spatial order rather than committing to a single exact curve.

5. Analysis

Why the method works. GaussianCity succeeds because it delegates different responsibilities to different parts of the pipeline. BEV projections provide globally consistent city geometry; visibility pruning ensures that the explicit representation is small enough to render efficiently; and the decoder restores the local context lost when points are sparsified. This division of labor is especially suitable for urban scenes, where global layout is highly structured and most appearance variation is instance- or category-specific.

Human evaluation. Figure 5 reports a user study on GoogleEarth and KITTI-360. Across both datasets, GaussianCity receives the highest average scores on perceptual quality, degree of 3D realism, and view consistency, matching the trend of the automatic metrics. This is important because speedups alone would be less meaningful if they were achieved by sacrificing multi-view realism.

Figure 4. **Qualitative comparison on KITTI-360.** In street-view scenes, GaussianCity better preserves facade structure and scene realism under perspective distortion.

Cross-dataset behavior. The gains on GoogleEarth and KITTI-360 also reveal how the representation transfers across viewing regimes. In aerial scenes, the main benefit is scalability: large visible extents make dense neural fields particularly expensive, so the 60× runtime reduction directly translates into better quality under a fixed compute budget. In street-view scenes, the advantage is more about structural precision. Because nearby facades and roadside objects occupy many pixels, pruning to a compact but visible point set leaves more decoder capacity for appearance refinement, which is consistent with the larger relative KID improvement on KITTI-360. Together, these results indicate that BEV-Point improves both speed and capacity allocation.

Failure modes and scope. The method also has clear boundaries. Because Gaussian geometry is inherited from BEV extrusion, fine geometric deviations that are absent in the BEV projections cannot be recovered purely by appearance prediction. Likewise, heavy occlusions or missing BEV annotations can remove points that would otherwise support plausible rendering. These limitations are consistent with the design goal of scalable generation from compact structured inputs rather than fully unconstrained geometry synthesis.

6. Conclusion

We presented GaussianCity, a generative 3D Gaussian splatting framework for unbounded city generation. The core idea is to make explicit Gaussian rendering scalable by combining a compact visibility-aware BEV-Point representation with a spatially aware decoder that restores structure after sparsification. The resulting system achieves strong fidelity on both GoogleEarth and KITTI-360 while substantially improving inference speed over prior NeRF-based urban generators. The results also indicate that explicit geometry and learned appearance can be balanced effectively across both aerial and street-view settings. This modularity is attractive for interactive simulation and navigation workloads that need both layout control and fast rendering. More broadly, the results suggest that city-scale generation does not require dense neural fields if geometry is organized carefully and the decoder is designed to exploit that structure.

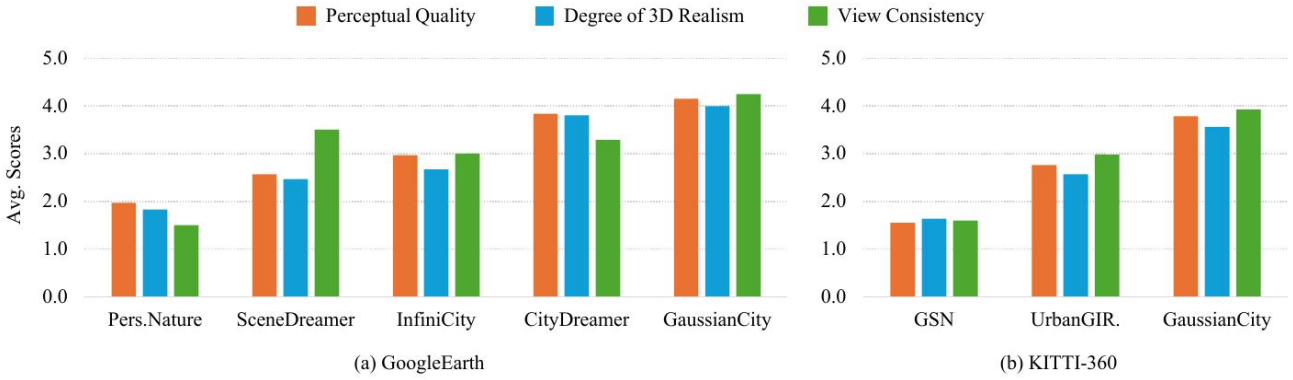

Figure 5. **User study on GoogleEarth and KITTI-360.** GaussianCity obtains the highest ratings across perceptual quality, 3D realism, and view consistency on both benchmarks.

References

[1] John Amanatides and Andrew Woo. A fast voxel traversal algorithm for ray tracing. In *Eurographics*, 1987. 3

[2] Mikolaj Binkowski, Danica J. Sutherland, Michael Arbel, and Arthur Gretton. Demystifying MMD GANs. 2018. 4

[3] Lucy Chai, Richard Tucker, Zhengqi Li, Phillip Isola, and Noah Snavely. Persistent Nature: A generative model of unbounded 3D worlds. 2023. 5

[4] Wanli Chen, Xinge Zhu, Guojin Chen, and Bei Yu. Efficient point cloud analysis using hilbert curve. 2022. 2, 3

[5] Zhaoxi Chen, Guangcong Wang, and Ziwei Liu. SceneDreamer: Unbounded 3D scene generation from 2D image collections. 45(12):15562–15576, 2023. 1, 2, 5

[6] Terrance DeVries, Miguel Ángel Bautista, Nitish Srivastava, Graham W. Taylor, and Joshua M. Susskind. Unconstrained scene generation with locally conditioned radiance fields. 2021. 5

[7] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. In *NIPS*, 2017. 4

[8] David Hilbert. Über die stetige abbildung einer linie auf ein flächenstück. *Mathematische Annalen*, 38: 459–460, 1891. 2, 3

[9] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of styleGAN. 2020. 3, 5

[10] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans. Gr.*, 42(4):139:1–139:14, 2023. 1, 2, 4

[11] Yiyi Liao, Jun Xie, and Andreas Geiger. KITTI-360: A novel dataset and benchmarks for urban scene understanding in 2D and 3D. 45(3):3292–3310, 2023. 4

[12] Chieh Hubert Lin, Hsin-Ying Lee, Willi Menapace, Menglei Chai, Aliaksandr Siarohin, Ming-Hsuan Yang, and Sergey Tulyakov. InfiniCity: Infinite-scale city synthesis. 2023. 2

[13] Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-GS: Structured 3D Gaussians for view-adaptive rendering. 2024. 2

[14] Michael Niemeyer and Andreas Geiger. GIRAFFE: representing scenes as compositional generative neural feature fields. 2021. 1, 2, 5

[15] Yuan Shen, Wei-Chiu Ma, and Shenlong Wang. SGAM: building a virtual 3D world through simultaneous generation and mapping. 2022. 1, 2, 5

[16] Stanislaw Szymanowicz, Christian Rupprecht, and Andrea Vedaldi. Splatter image: Ultra-fast single-view 3D reconstruction. 2024. 2

[17] Jiaxiang Tang, Zhaoxi Chen, Xiaokang Chen, Tengfei Wang, Gang Zeng, and Ziwei Liu. LGM: large multi-view Gaussian model for high-resolution 3D content creation. 2024. 2

[18] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer V3: simpler, faster, stronger. 2024. 2, 3

[19] Haozhe Xie, Zhaoxi Chen, Fangzhou Hong, and Ziwei Liu. CityDreamer: Compositional generative model of unbounded 3D cities. 2024. 1, 2, 4, 5

[20] Yinghao Xu, Zifan Shi, Wang Yifan, Hansheng Chen, Ceyuan Yang, Sida Peng, Yujun Shen, and Gordon Wetzstein. GRM: Large Gaussian reconstruction model for efficient 3D reconstruction and generation. *arXiv*, 2403.14621, 2024. 2

[21] Yuanbo Yang, Yifei Yang, Hanlei Guo, Rong Xiong, Yue Wang, and Yiyi Liao. UrbanGIRAFFE: Representing urban scenes as compositional generative neural feature fields. 2023. 1, 2, 5